



MANUAL TÉCNICO

Dessert Sacré — Plataforma Digital para Panaderías

Fecha: 2025

**Proyecto:
Dessert Sacré**

**Programa:
Técnico en desarrollo de software**

SENA Centro de biotecnología industrial

**Repositorio:
<https://github.com/Shalomoreno>**

Integrantes:

**Moreno S. Salome
González R. Laura S.
Benavides C. Maria Angelica
Torres M. Laura S.**

Palmira, Valle del Cauca — Colombia

1. INTRODUCCIÓN AL PROYECTO

1.1 Contexto y Motivación

Dessert Sacré nació como respuesta directa a una problemática directa en Palmira, Valle del Cauca, donde existe una fuerte cultura panadera a lo largo de décadas. Sin embargo, estas panaderías y reposterías operan en barrios y la gran mayoría son invisibles en el mercado digital. Estos negocios no suelen aparecer en buscadores, no tienen redes sociales, y administran sus pedidos de manera manual en cuadernos, por WhatsApp o de memoria. Se creó una plataforma web desarrollada con Flask (Python) y PostgreSQL diseñada para ofrecer a panaderías y reposterías de Palmira y sus municipios vecinos mediante el aplicativo una forma de gestionar sus pedidos de forma ordenada en conjunto de un sistemas de ventas en línea con un amplio catálogo de productos según el cliente lo necesite.

1.2 Validación de Mercado

El equipo realizó unas entrevistas a 12 panaderías de Palmira, desde negocios de barrio hasta los más reconocidos de la ciudad. Los resultados que se recolectaron fueron que 8 de las 12 panaderías entrevistadas mostraron interés real en implementar la solución planteada anteriormente, y más de la mitad reconoció la necesidad de una herramienta digital como la que plantea Dessert Sacré. Esta investigación sirvió para tomar decisiones de diseño y del desarrollo del aplicativo para que este quedara más acorde a las necesidades recolectadas.

1.3 Modelo de Negocio

Ofrecemos a las panaderías una página web o aplicación que les permite digitalizar su negocio, mostrar sus productos, gestionar pedidos y aumentar sus ventas. Resolvemos problemas como la falta de presencia digital, dificultad para organizar pedidos y poca visibilidad en el mercado. Ofrecemos un servicio tecnológico accesible, personalizable y fácil de usar. Satisfacemos la necesidad de modernización, organización y crecimiento del negocio.

1.4 Alcance del Manual

El presente Manual Técnico describe la arquitectura, el funcionamiento interno, la estructura del código fuente, la configuración del entorno de despliegue con Docker, el esquema de base de datos, la lógica de los módulos principales de la aplicación, las rutas de la API. Está dirigido al personal técnico o desarrolladores encargados del mantenimiento, actualización o implementación del sistema.

2. Arquitectura del Sistema

2.1 Visión General

Dessert Sacré sigue una arquitectura de aplicación web que es desplegada mediante contenedores Docker. Este proyecto se compone de tres capas siendo estas la capa de presentación, es decir el frontend utilizado tales como plantillas HTML/Jinja2 + CSS + JavaScript, la capa de lógica del aplicativo o sea Flask en Python y capa de base datos (PostgreSQL).

2.2 Diagrama de Componentes

Cliente (Navegador Web)

El usuario puede acceder a la aplicación a través de cualquier navegador. Estas páginas son renderizadas mediante el motor de plantillas Jinja2 y servidas como HTML. Además, el carrito de compras opera mediante sesiones del lado del servidor, gestionadas por Flask con cookies firmadas.

Servidor de Aplicación — Flask

El servidor de aplicación es Flask, que actúa como el núcleo de la aplicación, este gestiona el enrutamiento HTTP, la lógica del aplicativo, la autenticación de usuarios, el manejo de sesiones, el envío de correos electrónicos SMTP y la comunicación con la base de datos mediante psycopg2.

Base de Datos — PostgreSQL 15

La base de datos funciona a través de PostgreSQL, está almacena todos los datos de usuarios registrados y los códigos de verificación y recuperación de contraseña, así mismo la base de datos guarda información como pedidos, productos y categorías.

Infraestructura — Docker Compose

Por parte de la infraestructura, la aplicación y la base de datos se despliegan como servicios independientes por medio de Docker Compose. Este servicio web expone el puerto 5000 y el servicio 'db' (PostgreSQL 15) persiste sus datos en un volumen Docker nombrado.

2.3 Flujo de Petición HTTP

En el flujo de petición HTTP primeramente el navegador del usuario envía una petición HTTP al servidor Flask en el puerto 5000. A lo que Flask identifica la ruta (endpoint) correspondiente mediante su sistema de enrutamiento (@app.route), luego la función de vista ejecuta la lógica de negocio: valida la sesión, consulta la base de datos mediante psycopg2, procesa los datos y prepara el contexto de renderizado, ahí Jinja2 renderiza la plantilla HTML correspondiente, inyectando el contexto generado en el paso anterior y finalmente Flask devuelve la respuesta HTTP al cliente (HTML renderizado, JSON para las rutas de API, o una redirección HTTP).

2.4 Patrones de Diseño Aplicados

Patrón	Aplicación en Dessert Sacré
MVC (Model-View-Controller)	Separación entre lógica (app.py), plantillas (templates/) y datos (PostgreSQL)
Repository-like (parcial)	Función <code>get_db_connection()</code> centraliza el acceso a la BD
Session-based Auth	Autenticación y autorización mediante sesiones del lado del servidor
Factory Function	<code>crear_tabla()</code> inicializa el esquema de BD al arrancar la aplicación
Decorator Routing	<code>@app.route</code> define los endpoints de la aplicación

3. Stack Tecnológico

3.1 Dependencias del Proyecto

El archivo requirements.txt define todas las dependencias de Python con sus versiones:

Librería	Versión	Propósito
Flask	3.1.2	Framework web principal. Enrutamiento, contexto de solicitud, sesiones y plantillas.
psycopg2-binary	2.9.11	Driver PostgreSQL para Python. Permite ejecutar consultas SQL y gestionar conexiones.
Werkzeug	3.1.3	Utilidades HTTP subyacentes de Flask. Proporciona generate_password_hash y check_password_hash.
Jinja2	3.1.6	Motor de plantillas HTML. Permite lógica de presentación en los archivos .html.
itsdangerous	2.2.0	Firma criptográfica de cookies de sesión. Garantiza la integridad de los datos de sesión.
click	8.3.0	Dependencia de Flask para la interfaz de línea de comandos.
blinker	1.9.0	Sistema de señales de Flask. Habilita el uso de señales de ciclo de vida.
MarkupSafe	3.0.3	Escape seguro de HTML en plantillas Jinja2 para prevenir XSS.
colorama	0.4.6	Soporte de colores ANSI en terminal para logs de Flask en Windows.

3.2 Infraestructura y Entorno

Componente	Versión	Rol
Python	3.13	Entorno de ejecución del servidor Flask
PostgreSQL	15	Sistema de gestión de base de datos relacional
Docker	Compatible con Compose v3.9	Contenedorización del entorno de ejecución
Docker Compose	v3.9	Orquestación multi-contenedor (web + db)
SMTP Gmail	Puerto 587 / STARTTLS	Servicio de envío de correos transaccionales

3.3 Lenguajes y Tecnologías del Frontend

- HTML5 con motor de plantillas Jinja2 para el renderizado en el servidor.
- CSS3 (hojas de estilo propias en el directorio static/).
- JavaScript del lado del cliente para interacciones dinámicas: manejo del carrito vía fetch() a las rutas /api/cart, actualización de cantidades y eliminación de ítems sin recarga completa de página.
- No se utiliza ningún framework de frontend (React, Vue, Angular). La aplicación es server-side rendered (SSR) con enriquecimiento JavaScript mínimo.
- Uso de bootstrap-5.3.8 para la creación e interacción de la barra de navegación y el uso de responsive.

4. Estructura del Repositorio

4.1 Arbol de Directorios Principal

- static
 - img
 - M2.png
 - macarons.jpg
 - menu1.png
 - muffins.jpg
 - orejas.jpg
 - pan aliñado.jpg
 - pan coco.jpg
 - pan crema.jpg.png
 - pan huevo.jpg
 - Pan integral.jpg
 - pan piñita.jpg
 - Pan salchicha.jpg
 - pan.jpg
 - panaderia1.jpeg
 - panaderia2.jpeg
 - panaderia3.jpeg
 - pancacho.jpg
 - pandebono.jpg
 - pasteleria1.jpeg
 - pasteleria2.jpeg
 - pasteleria3.jpeg
 - pera.jpg
 - postre.jpg
 - reposteria1.jpeg
 - reposteria2.jpeg
 - reposteria3.jpeg
 - salon.png
 - sbbloque1.jpg
 - buñuelo.jpg
 - cafe con leche.jpeg
 - cafe.jpg
 - cede.png
 - Chocolate.jpeg
 - coca-cola.jpeg
 - cupcakes.jpg
 - donas.jpg
 - dS1.mp4
 - empanada de cambray.jpg
 - fondo1.jpeg
 - fondo2.jpeg
 - galleta cuca.jpg
 - Inicio.mp4
 - Jugo de guanabana.jpeg
 - jugo de guayaba.jpeg
 - jugo de mango.jpeg
 - jugo de maracuya.jpeg
 - jugo de mora.jpeg
 - Lara.png
 - Laura.jpeg
 - Limonada.jpeg
 - LOGO.png
 - LOGO1.png
 - M2.png
 - macarons.jpg
 - menu1.png

4.2 Archivos de Configuración Clave

config.py

Define la clase Config que centraliza todas las configuraciones de la aplicación. Lee las variables de entorno con `os.environ.get()` y usa valores por defecto para desarrollo local. Contiene: `SECRET_KEY` para la firma de sesiones, `DB_CONFIG`

G con los parámetros de conexión a PostgreSQL, `EMAIL_USER` y `EMAIL_PASS` con las credenciales SMTP de Gmail.

docker-compose.yml

Define dos servicios: 'web' (la aplicación Flask) construida desde el Dockerfile local, y 'db' (PostgreSQL 15 desde imagen oficial). El servicio 'web' depende del servicio 'db' mediante `depends_on`, expone el puerto 5000 al host, y monta el directorio actual en /app del contenedor para desarrollo con recarga en caliente. La base de datos persiste en el volumen Docker 'postgres_data'.

Dockerfile

Define la imagen Docker del servicio web. Típicamente: imagen base de Python, copia de `requirements.txt` e instalación de dependencias con pip, copia del código fuente, exposición del puerto 5000 y comando de arranque de la aplicación Flask.

5. Esquema de Base de Datos

5.1 Tablas Implementadas

La función **crear_tabla()** en app.py inicia la base de datos al arrancar el aplicativo como si las tablas no existieran aún (CREATE TABLE IF NOT EXISTS). Actualmente el sistema cuenta con dos tablas en producción:

5.1.1 Tabla: registro

Almacena los datos de todos los usuarios registrados en la plataforma.

Columna	Tipo	Restricciones	Descripción
id	SERIAL	PRIMARY KEY	Identificador único autoincremental del usuario
primer_nombre	VARCHAR(100)	NOT NULL	Primer nombre del usuario
segundo_nombre	VARCHAR(100)	NULL permitido	Segundo nombre del usuario (opcional)
primer_apellido	VARCHAR(100)	NOT NULL	Primer apellido del usuario
segundo_apellido	VARCHAR(100)	NULL permitido	Segundo apellido del usuario (opcional)
correo	VARCHAR(150)	UNIQUE, NOT NULL	Correo electrónico. Llave natural y campo de autenticación
password	TEXT	NOT NULL	Hash de la contraseña generado con Werkzeug (PBKDF2)
telefono	VARCHAR(20)	NULL permitido	Número de teléfono de contacto del usuario
direccion	TEXT	NULL permitido	Dirección de entrega o domicilio del usuario
codigo_verificacion	VARCHAR(6)	NULL permitido	Código de 6 dígitos para verificación del correo
verificado	BOOLEAN	DEFAULT FALSE	Indica si el usuario ha verificado su correo electrónico

5.1.2 Tabla: recuperacion

Almacena los códigos temporales de recuperación de contraseña.

Columna	Tipo	Restricciones	Descripción
id	SERIAL	PRIMARY KEY	Identificador único del registro de recuperación
correo	VARCHAR(150)	REFERENCES registro(correo)	Llave foránea al correo del usuario en la tabla registro
codigo	VARCHAR(6)	NOT NULL	Código de 6 dígitos generado aleatoriamente
expiracion	TIMESTAMP	DEFAULT NOW() + 15 min	Marca de tiempo de expiración del código (15 minutos)
usado	BOOLEAN	DEFAULT FALSE	Indica si el código ya fue consumido

5.2 Deuda Técnica: Pedidos en Memoria

En la versión actual, los pedidos se almacenan en el diccionario en memoria `pedidos_guardados = {}` dentro del proceso Flask. Esto significa que los pedidos se pierden al reiniciar el servidor y no son accesibles desde múltiples instancias del proceso. Para una versión productiva, se debe implementar una tabla 'pedidos' en PostgreSQL con la siguiente estructura sugerida:

Columna	Tipo	Descripción
id	SERIAL PRIMARY KEY	Identificador único del pedido
referencia	VARCHAR(20) UNIQUE NOT NULL	Código de referencia del pedido (ej: PED-AB12CD34)
correo_usuario	VARCHAR(150) REFERENCES registro(correo)	Correo del usuario que realizó el pedido
total	NUMERIC(12,2) NOT NULL	Monto total del pedido

Columna	Tipo	Descripción
metodo_pago	VARCHAR(50)	Método de pago seleccionado (nequi, banco, efecty)
fecha	TIMESTAMP DEFAULT NOW()	Fecha y hora de creación del pedido
estado	VARCHAR(30) DEFAULT 'pendiente'	Estado del pedido (pendiente, confirmado, entregado)

5.3 Seguridad en la Capa de Datos

- Todas las consultas SQL utilizan consultas parametrizadas (placeholders %s de psycopg2), eliminando el riesgo de inyección SQL.
- Las contraseñas nunca se almacenan en texto plano. Se usa generate_password_hash() de Werkzeug, que aplica el algoritmo PBKDF2 con sal aleatoria.
- La verificación de contraseñas se realiza con check_password_hash(), que compara de forma segura sin exponer el hash.
- Los códigos de verificación y recuperación son de 6 dígitos generados con random.randint(). Para mayor seguridad en producción se recomienda usar secrets.randbelow().

6. Módulos y Rutas de la Aplicación

6.1 Módulo de Registro de Usuarios

El registro de usuarios sigue un flujo de dos pasos: captura de datos y verificación del correo electrónico.

Ruta	Método	Función	Descripción
/register	GET	register()	Renderiza el formulario de registro. Limpia sesiones previas de verificación.
/guardar	POST	guardar()	Procesa el formulario: valida campos, verifica correo duplicado, hashea password, inserta usuario, envía código.
/verify	GET/POST	verify()	GET: renderiza el formulario de código. POST: verifica el código y marca al usuario como verificado.
/reenviar_codigo	GET	reenviar_codigo()	Genera y envía un nuevo código. Límite: 3 reenvíos por sesión.

Lógica clave de guardar()

- Verifica que primer_nombre, primer_apellido, correo y password estén presentes.
- Consulta si el correo ya existe en la tabla registro (previene duplicados).
- Aplica generate_password_hash() al password antes de insertar.
- Genera un código de 6 dígitos con random.randint(100000, 999999).
- Inserta el usuario con verificado=FALSE y llama a enviar_codigo().
- Guarda session['correo_verificacion'] y redirige a /verify.

6.2 Módulo de Autenticación

Ruta	Método	Función	Descripción
/login	GET/POST	login()	GET: renderiza login. POST: verifica credenciales, distingue admin vs usuario, gestiona sesión.
/logout	GET	logout()	Limpia toda la sesión con session.clear() y redirige a la raíz.

Ruta	Método	Función	Descripción
/dashboard	GET	dashboard()	Vista del dashboard del usuario. Requiere sesión activa.

Credenciales de Administrador

El administrador se autentica con credenciales hardcodeadas en app.py (ADMIN_EMAIL y ADMIN_PASSWORD). Al autenticarse, se establece `session['admin'] = True` y `session['usuario'] = 'Administrador'`. Para producción, estas credenciales deben migrarse a variables de entorno y la contraseña debe estar hasheada.

1. Introducción al Proyecto

- 1.1 Contexto y Motivación
- 1.2 Validación de Mercado
- 1.3 Propuesta de Valor
- 1.4 Modelo de Negocio
- 1.5 Alcance del Manual

2. Arquitectura del Sistema

- 2.1 Visión General
- 2.2 Diagrama de Componentes
 - Cliente (Navegador Web)
 - Servidor de Aplicación — Flask
 - Base de Datos — PostgreSQL
 - Infraestructura — Docker Compose
- 2.3 Flujo de Petición HTTP
- 2.4 Patrones de Diseño Aplicados

3. Stack Tecnológico

- 3.1 Dependencias del Proyecto
- 3.2 Infraestructura y Entorno
- 3.3 Lenguajes y Tecnologías del Frontend

4. Estructura del Repositorio

- 4.1 Árbol de Directorios Principal
- 4.2 Archivos de Configuración Clave

5. Esquema de Base de Datos

5.1 Tablas Implementadas

5.1.1 Tabla: registro

5.1.2 Tabla: recuperacion

5.2 Deuda Técnica: Pedidos en Memoria

5.3 Seguridad en la Capa de Datos

6. Módulos y Rutas de la Aplicación

6.1 Módulo de Registro de Usuarios

6.2 Módulo de Autenticación

POR HACER:

6.3 Módulo de Carrito de Compras

6.4 Módulo de Pasarela de Pagos

6.5 Módulo de Recuperación de Contraseña

6.6 Módulo de Administración

7. Seguridad

7.1 Medidas de Seguridad Implementadas

7.2 Vulnerabilidades Identificadas y Recomendaciones

8. Sistema de Correo Electrónico

8.1 Configuración SMTP

8.2 Función enviar_codigo()

8.3 Casos de Uso del Servicio de Correo

8.4 Limitaciones y Mejoras Sugeridas

9. Despliegue con Docker

9.1 Arquitectura Docker

9.2 Variables de Entorno del Servicio Web

9.3 Variables de Entorno del Servicio DB

9.4 Persistencia de Datos

9.5 Diferencia: host.docker.internal vs Nombre de Servicio

9.6 Comandos de Operación

10. Deuda Técnica y Hoja de Ruta

10.1 Deuda Técnica Identificada

10.2 Mejoras Sugeridas por Versión

11. Glosario Técnico